

Security Audit

Kron (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Compiled-Artifact Review	8
Security Rating	10
Intended Covenant Functions	11
Code Quality	15
Audit Resources	15
Severity Definitions	16
Status Definitions	17
Audit Findings	18
Centralisation	56
Conclusion	57
Our Methodology	58
Disclaimers	60
About Hashlock	61

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Kron team partnered with Hashlock to conduct a security audit of their covenants. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed covenants are secure.

Project Context

KRON is a Kaspas-based decentralized exchange (DEX) and token launchpad designed to enable users to swap tokens and launch new tokens within the Kaspas ecosystem. The platform focuses on browser-based, user-signed transactions, with trades built and signed locally rather than relying on a centralized transaction-signing service.

Project Name: Kron

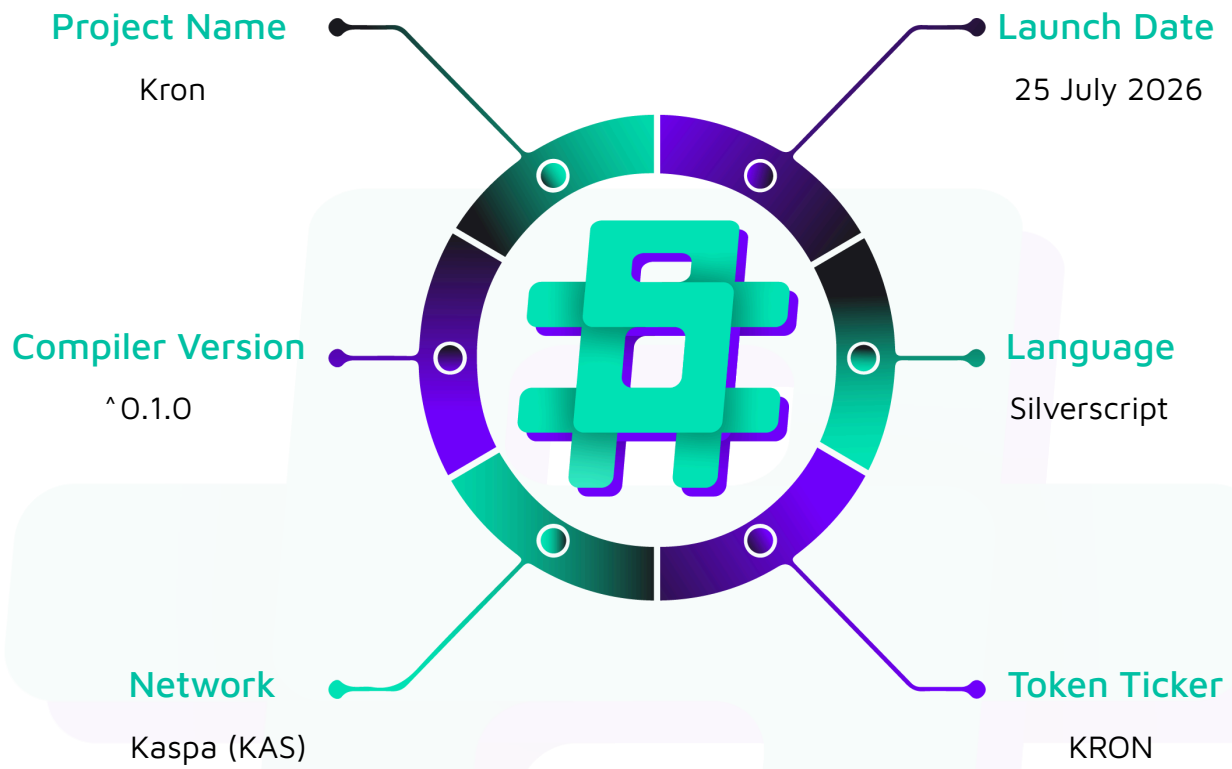
Project Type: Defi

Compiler Version: ^0.1.0

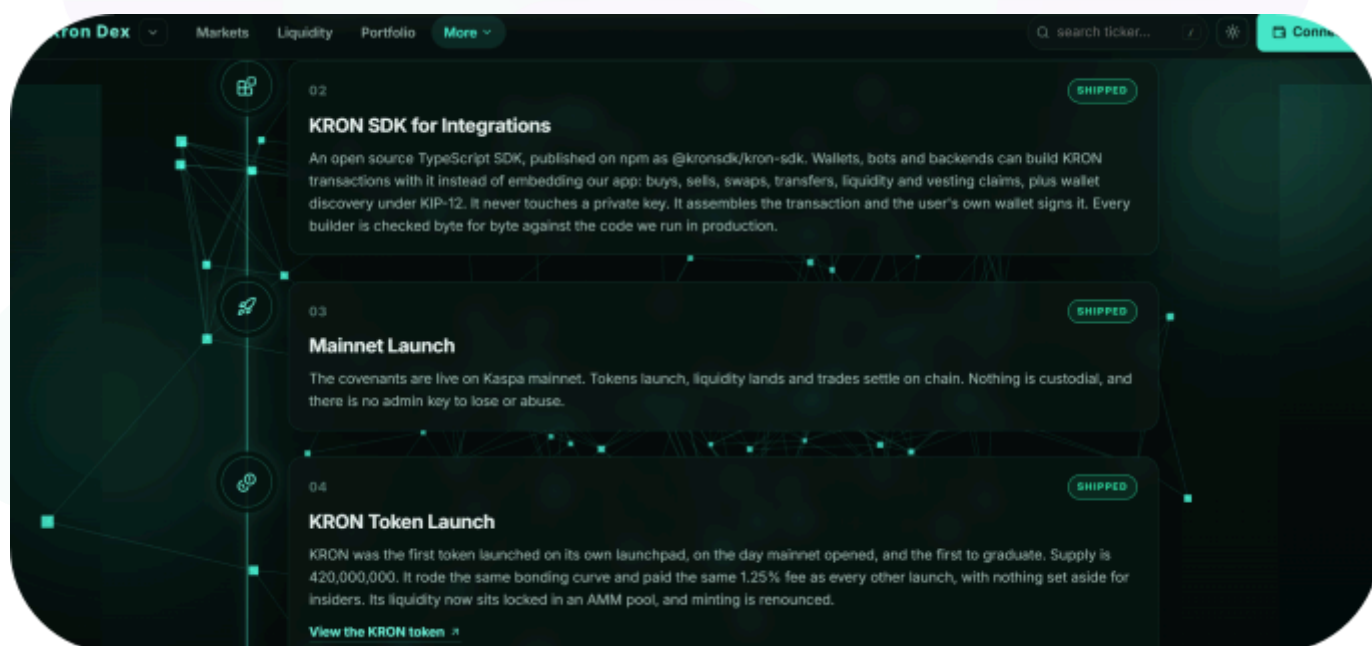
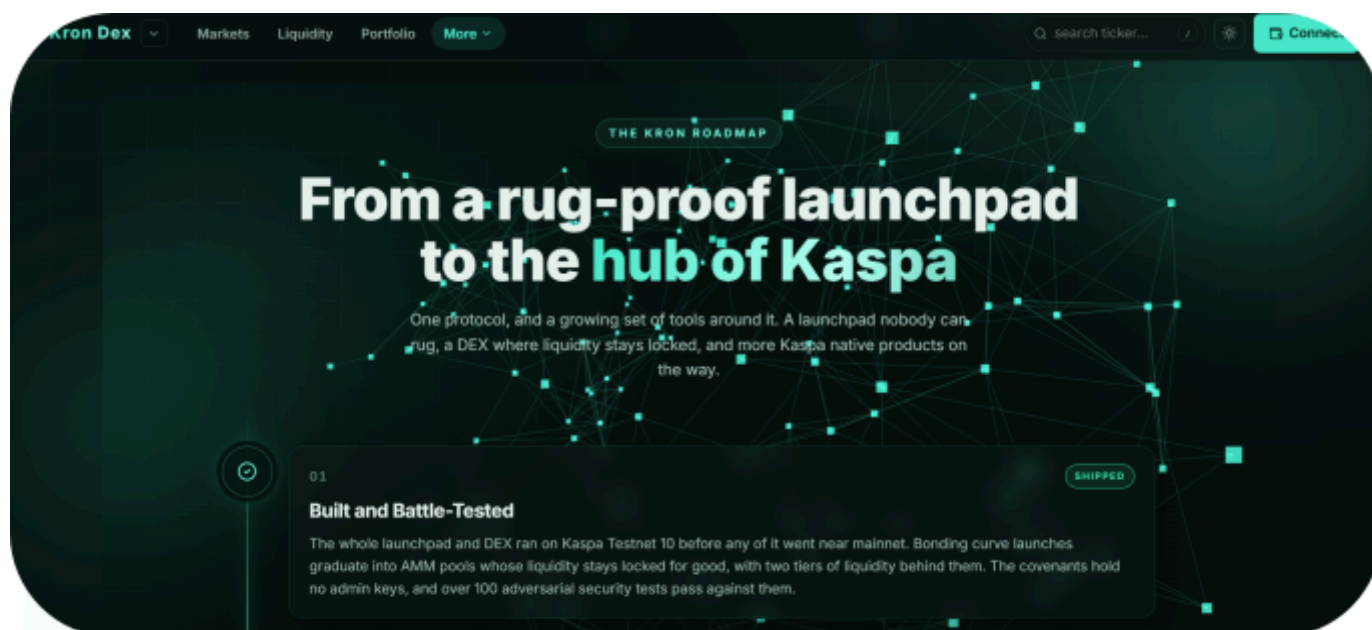
Website: <https://kron.technology/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Silverscript code within the Kron project, the scope of work included a comprehensive review of the covenants listed below. We tested the covenants to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Kron Covenants
Network	Kaspa
Language	Silverscript
Audit Date	September, 2026
Covenant 1	covenants/native/curve_cp.sil
Covenant 2	covenants/native/buy_order.sil
Covenant 3	covenants/native/amm_pool_cp_v3.sil
Covenant 4	covenants/native/kcc20.sil
Audited GitHub Commit Hash	a0de887373726fde47d3b6b7850f930c19db1da5
Fix Review GitHub Commit Hash	c99a297bbe85554a4bfd02fc9d1cc90488bacbfa

Note: Hashlock audited the code at the "Audited GitHub Commit Hash", and the findings in this report pertain to that commit. For the "Fix Review GitHub Commit Hash", Hashlock verified the resolution status of the identified findings.

This review covers only the code at the frozen commit listed above. It makes no statement about tokens deployed under earlier schemas, which hold live funds on mainnet and were not reviewed. This engagement was performed under significant platform constraints. SilverScript is a pre-1.0 language with no static analysis, fuzzing, or other tooling, no accumulated vulnerability corpus, and limited tooling beyond a public testnet, and the Kaspa covenant opcodes it relies on were activated on mainnet only weeks before this review. A clean result should be read as no critical issues found under these conditions, not as a guarantee that the code is safe.

Compiled-Artifact Review

Because SilverScript is a pre-1.0 language and a correct source guard does not guarantee the emitted script preserves it, this engagement included a targeted review at the compiled-artifact level, scoped to five named items. Each was checked against the emitted script produced by the pinned silverc compiler. All five passed with no additional findings.

1. **Reproducible build.** The schema hash reproduces at the pinned silverc commit, with byte-identical reproduction confirmed against all fifteen archived manifests.
2. **Dispatch by declaration order.** Confirmed on both live selector maps. The compiled selector order matches the source declaration order in `curve_cp`, `amm_pool_cp_v3`, and `buy_order`, with each branch confirmed by its first emitted guard.
3. **ZERO_COVID guards at opcode level.** The `tokenCovid` guard on `init` and the `lpCovid` guards on `addLiquidity` and `removeLiquidity` are each present in the emitted bytes, and `batchBuy`'s per-order guard is correctly unrolled four times.
4. **batchBuy allocation.** The pro-rata allocation block is present in the emitted script, with the pre-remainder bound emitted at the end of the block.
5. **requireOwnTokenReserveInput.** The guard is present at all four call sites in the emitted script.

Off-chain resolver. The template-to-selector resolution path in `backend/templateVersions.mjs`, including its memoisation, was reviewed. The template-pinning gate is green, and the compiled output reproduces byte-identically against all fifteen archived manifests.

Transient mass ceiling. All ten entrypoint shapes sit under the 1,000,000 consensus mass limit, with a worst case of 84.4 percent. The applicable ceiling was confirmed against rusty-kaspa consensus source. The mass limit rises from 500,000 to 1,000,000 when the Toccata upgrade activates, and the covenant instructions `KRON` is built from are switched on by that same upgrade, so `KRON` can only execute in a context where the limit is 1,000,000.

Compiled-level coverage by endpoint. This review was scoped to the five items above and is not a full compiled-level verification of every endpoint. Coverage is as follows.

- `curve_cp.sil`, `init`, `buy`, `sell`, `graduate`, `batchBuy`. Dispatch verified at the compiled level for each. On `init`, the `tokenCovid` guard verified in emitted bytes. On `batchBuy`, the per-order `ZERO_COVID` guard, the pro-rata allocation, and the pre-remainder bound verified in emitted script.
- `buy_order.sil`, `settle`, `refund`. Dispatch verified at the compiled level for each.
- `amm_pool_cp_v3.sil`, `swapKasForToken`, `swapTokenForKas`, `addLiquidity`, `removeLiquidity`, `bindLp`. Dispatch verified at the compiled level for each. On `addLiquidity` and `removeLiquidity`, the `lpCovid` guard verified in emitted bytes. `requireOwnTokenReserveInput` verified at its four call sites.
- `kcc20.sil`, `transfer`. Reviewed at source level only, not among the compiled-artifact targets.

Findings concerning runtime VM behaviour (see L-11) were not exercised against a live node.

Security Rating

After Hashlock's Audit, we found the covenants to be **"Secure"**. The covenants all follow simple logic, with correct and detailed ordering. The compiled artifacts for the reviewed targets were verified against source at the emitted-script level, as set out in the Compiled-Artifact Review section.

**Not Secure****Vulnerable****Secure****Hashlocked**

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's covenant functionality is presented in the [Intended Covenant Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

13 Low severity vulnerabilities

10 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Covenant Functions

Claimed Behaviour	Actual Behaviour
covenants/native/curve_cp.sil • init: One-time launch. ◦ Binds the curve to a token (covenant id A), mints the full supply (inventory + dev allocation), and commits the token reserve into curve state. • buy: Synchronous buy. ◦ Buyer pays KAS (kasIn), receives tokens (tokenOut) at the constant-product price. ◦ Increases the curve's KAS reserve and decreases the token reserve. ◦ Supports merge-in-buy (buyer's existing tokens merge into one output). ◦ Pays creator/platform/dev fees. • sell: Synchronous sell. ◦ Seller folds tokens (tokenIn) into the inventory, receives KAS (kasOut) at the constant-product price. ◦ Shrinks the curve's KAS reserve, grows the token reserve. ◦ Returns unsold change as a separate output. ◦ Pays creator/platform/dev fees. • graduate: ◦ One-way transition when the raise reaches graduationKas. ◦ Locks the curve (graduated = true), seeds an AMM pool with ~95% of the raised KAS	Covenant achieves this functionality.

and the leftover token inventory, pays the platform graduation fee. ● batchBuy: Asynchronous batch settlement. ○ Settles 2-4 pre-funded buyer orders at one aggregate constant-product price. ○ Splits tokens pro-rata by each order's kasIn contribution. ○ Pays aggregated creator/platform/dev fees.	
covenants/native/buy_order.sil ● settle: ○ Validates a buyer's payout when their order is consumed in a batchBuy. ○ Checks the buyer's identity, minimum token amount, and output Kron. ○ Uses slot to identify which token output is the buyer's payout. ● refund: ○ Cancels a pending order and returns the full escrow to the buyer's P2PK address. ○ Requires a co-present buyer P2PK input (only the buyer can refund). ○ One order per transaction.	Covenant achieves this functionality.
covenants/native/amm_pool_cp_v3.sil ● swapKasForToken ○ Buy on the DEX. The trader pays kasIn plus fees and receives tokenOut from the pool reserve at the constant-product price. ● swapTokenForKas ○ Sell on the DEX, fractional. The trader folds part of a	Covenant achieves this functionality.

token piece into the pool, receives kasOut minus fees, and gets the unsold

remainder back as a single presence-owned change output.

- addLiquidity
 - Deposit dKas and dToken at the current ratio with no price impact, and receive dShares of the LP-share token L moved out of the pool's inventory rather than minted.
- removeLiquidity
 - Return dShares to the pool inventory and withdraw a strictly proportional dKas and dToken, both floored.
- bindLp
 - One-time, permissionless binding, run once after graduation and before any addLiquidity.
 - Genesis-mints the LP-share token L at a fixed supply (MAX_SHARES) with no minter branch, so L can never be minted again.
 - Binds L's covenant id into the pool state (lpCovid moves from ZERO to the real id), which is what enables addLiquidity and removeLiquidity.
 - Seeds the pool's L inventory with exactly the issuable amount (MAX_SHARES – lockedShares), owned by the pool covenant. The locked floor (lockedShares) is never minted and exists only as a counter and the removeLiquidity floor guard.
 - Leaves the KAS and token reserves and totalShares unchanged, and must not move the

pool's token reserve.	
covenants/native/kcc20.sil • transfer ◦ The sole entry point and the only way a token UTXO moves.	Covenant achieves this functionality.

Code Quality

This audit scope involves the covenants of the Kron project, as outlined in the Audit Scope section. The code is very well commented on and is clearly and consistently styled. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Kron project covenant code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] covenants/native/curve_cp.sil#batchBuy - batchBuy pins the inventory continuation to an absolute dust constant

Description

Every covenant-owned token UTXO carries a small amount of native KAS (the "carrier value") to stay alive on-chain for storage-mass purposes.

The `buy`, `sell`, and `graduate` entrypoints all enforce this with a relative check: `require(tx.outputs[...].value >= tx.inputs[INVENTORY_IN].value)`, which indicates that the inventory output's carrier KAS must be at least as much as the inventory input's. This was added to prevent "shaving", where an attacker pockets the carrier KAS as a miner fee.

However, `batchBuy` was supposed to get the same fix but instead pins the inventory output to an absolute constant: `require(tx.outputs[...].value == ORDER_TOKEN_DUST)`. The stated justification is that "`batchBuy` can pin an exact figure because each order pre-funds a known escrow", but that's true of the buyer outputs (line 608), not the inventory output, which is a continuation of a pre-existing UTXO whose carrier value was chosen by whoever traded last and can be anything above dust.

Impact

Any inventory carrier value above 0.5 KAS is silently lost to the miner fee on the next `batchBuy`, the same defect the relative `>=` fix was written to close in the other three entrypoints. The exact output count check (`== count + 5`, line 506) prevents adding a skimming output, so the excess goes to the miner fee.

The inventory can get above dust via `buy` (line 309) and `sell` (line 378), which both use `>=` and allow the builder to pad the carrier value. For example, if the inventory carries 100 KAS, a single `batchBuy` shaves 99.5 KAS to the miner fee. The value is lost from the

covenant-owned inventory, repeatable on every batch until the carrier value is shaved down to dust.

Recommendation

Change line 576 to the same relative form the other three entrypoints use:

```
require(tx.outputs[OpCovOutputIdx(tokenCovid, 0)].value >= tx.inputs[INVENTORY_IN].value);
```

Status

Resolved

Low

[L-01] covenants/native/curve_cp.sil#init - `init` bounds only the fee bps, not `vKas` / `curve value` / `poolLockedShares`

Description

`init` is the one-time launch function where the creator binds the curve to a token and mints the full supply. The constructor parameters, `vKas` (virtual reserve), `graduationKas` (raise target), `poolLockedShares` (AMM floor shares), and the fee bps, are baked into the curve's P2SH address at deploy time and can never be changed. Every trading entrypoint (`buy`, `sell`, `batchBuy`) checks these params at runtime and reverts if they're out of range.

The problem is that `init` only checks the fee bps (via `fees0k()`) but not `vKas`, the curve's seed KAS value, or `poolLockedShares`. So a curve deployed with out-of-range `vKas` passes `init` cleanly, mints the entire token supply, and then reverts on every single trade because `buy/sell/batchBuy` all require `vKas ∈ [1, MAX_UNITS]`.

By the time the misconfiguration surfaces, the supply is already minted, and the launch fee is already burned. Since the parameters live in the address, no affected token can be patched or migrated.

Impact

The token is permanently broken depending on which parameter is out of range:

- out-of-range `vKas` means every `buy/sell/batchBuy` reverts forever (the token is dead, the launch fee is burned, the inventory is frozen)
- A curve seed value that isn't a multiple of `SCALE` or exceeds `MAX_KAS` has the same effect
- out-of-range `poolLockedShares` means the token trades but can never graduate (the raised KAS can never seed a pool and the inventory is stranded on the curve forever).

Recommendation

Validate all three in `init`:

```
require(vKas >= 1 && vKas <= MAX_UNITS);  
  
require(poolLockedShares >= 1 && poolLockedShares <= MAX_LOCKED_SHARES);  
  
int seed = tx.inputs[this.activeInputIndex].value;  
  
require(seed >= 0 && seed <= MAX_KAS && seed / SCALE * SCALE == seed);
```

Additionally, `graduationKas` should be checked to be `>= 0` and `<= MAX_KAS`.

Status

Resolved

[L-02] covenants/native/curve_cp.sil#batchBuy - `count == 2 batchBuy` wedge at current constants

Description

`batchBuy` settles 2-4 pre-funded buyer orders at once. For the transaction to be consensus-valid, the total input value must cover the total output value (Kaspa's conservation rule). The inventory output is pinned to an absolute constant (`ORDER_TOKEN_DUST = 0.5 KAS`) by line 576, while each order input contributes `ORDER_TX_FEE = 0.2 KAS` of spendable value.

In a `count == 2 batch`, the miner fee resolves to `inventoryInputValue + 2 × 0.2 KAS - 0.5 KAS = inventoryInputValue - 0.1 KAS`. For this to be non-negative, the inventory UTXO must carry at least 0.1 KAS of native carrier value. However, `init` does not constrain the inventory output's carrier value; it only pins the curve output's value.

Furthermore, the first-party `init` builder defaults to `tokenDust = 1000n sompi` (`web/src/native/curveCpTx.ts:198`), so a freshly initialized token can ship an inventory UTXO worth 0.000001 KAS, well below the 0.1 KAS threshold.

Impact

If the inventory starts below 0.1 KAS, the first `count == 2 batchBuy` is consensus-infeasible: the transaction's outputs exceed its inputs.

With that said, `count == 3` and `count == 4` are unaffected (their miner fee threshold is `<= 0` at current constants), so this is not a permanent wedge: a single buy ratchets the inventory up via the `>=` continuation, after which `count == 2` becomes feasible. But until that first buy lands, the token's first batch settlement cannot use exactly two orders.

Recommendation

Change line 576 to `>= tx.inputs[INVENTORY_IN].value` and update `buildCpBatchBuy` (`web/src/native/curveCpTx.ts:394`) to emit `continuationValue(dust, inventory.value)`. With `>=` in place, any non-negative miner fee is achievable, and the wedge disappears regardless of the inventory's starting value.

Status

Resolved



[L-03] `covenants/native/curve_cp.sil#init` - `creatorIdentifier` is never bound to `creatorFeeOwner`

Description

The curve constructor takes two separate parameters that should refer to the same person: `creatorFeeOwner` (a pubkey that receives the creator's share of trade fees) and `creatorIdentifier` (the creator's x-only pubkey as bytes, which owns the dev allocation).

The constructor comment says they're equal, but no check enforces this. `init` authorizes the caller against `creatorFeeOwner` (line 185). At the same time, the dev allocation is presence-owned by `creatorIdentifier` (line 221), so a curve can be deployed where the dev allocation goes to a completely different party than the one who signed the init and receives the fees.

Impact

A registry or UI mismatch silently mis-delivers the dev allocation to whatever `creatorIdentifier` was baked in.

Recommendation

Enforce the documented relationship in `init` (compare `creatorIdentifier` against `creatorFeeOwner`), or drop `creatorIdentifier` and derive the dev-allocation owner from `creatorFeeOwner`.

Status

Resolved

[L-04] covenants/native/curve_cp.sil#buy - Buyer / change output owners are unconstrained

Description

When a buyer buys tokens via `buy`, the covenant checks the buyer's output amount, ownership type, and minter flag, but never checks who the output belongs to (`buyerOut.ownerIdentifier` is not constrained). The same gap exists in `sell` for the seller's unsold-change output.

By contrast, `batchBuy` binds the recipient explicitly from the authenticated order state (line 606), and `refund` forces output 0 to the buyer's P2PK address. So `buy` and `sell` are the only token-moving entry points where the covenant doesn't pin the recipient; the transaction builder chooses it freely.

Impact

Both entry points require the trader's signature (their P2PK input funds the trade), so the attacker can't redirect tokens without the user signing. However, if a wallet blind-signs, or a malicious relayer/front-end passes their own pubkey instead of the buyer's, the bought tokens or unsold change go to an attacker-chosen address.

Recommendation

Bind the owner explicitly, as `batchBuy` already does, pass the expected recipient identifier, and add `require(buyerOut.ownerIdentifier == ...) / require(traderChangeOut.ownerIdentifier == ...)`.

This moves `buy/sell` from "the user's signature is the only barrier" to "the covenant forces the recipient, and the signature authorizes it", matching `refund`'s existing pin.

Status

Resolved

[L-05] covenants/native/curve_cp.sil#sell - sell allows `newKas == 0` which is rejected by Kaspas consensus

Description

`sell` requires only `newKas >= 0` (line 356), allowing a seller to withdraw all real KAS from the curve in a single sell. The curve continuation output would then carry `value == 0`.

However, Kaspas consensus unconditionally [rejects any transaction output](#) with `value == 0` via `check_transaction_output_value_ranges`, which returns `TxRuleError::TxOutZero(i)`, a hard consensus rule enforced at block acceptance time. The covenant's `require(newKas >= 0)` passes, but the node rejects the transaction.

Impact

A seller who sets `kasOut = realKas` (buying all KAS) produces a curve continuation output with `value == 0`. The covenant authorizes the transaction, but Kaspas consensus rejects it with `TxOutZero`, causing the sell to fail.

Recommendation

Tighten `require(newKas >= 0)` to `require(newKas >= 1)` in `sell`, matching the pattern already used in `swapTokenForKas` (`covenants/native/amm_pool_cp_v3.sil:253`). This makes the covenant's constraint match what consensus will actually accept, so a sell that would produce a 0-value output is rejected at the script level rather than failing at the node.

Status

Resolved

[L-06] covenants/native/curve_cp.sil#graduate - poolLockedShares and the pool Kron's lockedShares are independent parameters that nothing binds together

Description

The `graduate` entrypoint pins the pool's seeded share count to a curve constructor argument with `require(pool.totalShares == poolLockedShares)` at `curve_cp.sil:445`. The pool's own `lockedShares` is a different constructor argument, baked into `poolPrefix` and `poolSuffix` and covered only by `poolKronHash`, because it sits outside the pool's state window. Nothing forces the two to agree. They are reconciled only by the builder, at `web/src/curveConfig.ts:206` and `web/src/native/curveCpTx.ts:482-499`.

If the baked `lockedShares` exceeds the seeded `totalShares`, `sharesOk()` at `amm_pool_cp_v3.sil:101-104` returns false. All five entrypoints call it, at `:183`, `:241`, `:301`, `:379` and `:460`, so every one reverts.

If the two merely differ, `require(totalShares == lockedShares)` in `bindLp` at `:462` can never be satisfied and the LP token is never mintable.

Neither state is recoverable. The `addLiquidity` and `removeLiquidity` entrypoints both gate on `require(lpCovid != ZERO_COVID)` at `:295` and `:372`, so `totalShares` cannot move away from the graduation-seeded value while `L` is unbound.

Vulnerability Details

```
// covenants/native/curve_cp.sil:445 - the curve's parameter
require(pool.totalShares == poolLockedShares);

// covenants/native/amm_pool_cp_v3.sil:101-104 - the pool's own, baked
separately
function sharesOk(): bool {
    return totalShares >= 1 && totalShares <= MAX_SHARES
        && lockedShares >= 0 && lockedShares <= totalShares;
}
```

Impact

The graduate call itself succeeds, because the curve never evaluates `sharesOk()`. A deploy-time mismatch therefore moves roughly 95 percent of the raised KAS plus the entire leftover inventory into a pool that no entrypoint can spend, permanently, with no recovery path.

Recommendation

Derive both values from one source in the deploy pipeline, and assert at launch time that the compiled pool Kron's `lockedShares` equals the curve's `poolLockedShares` before the curve address is published. Add a `verify:Krons` case asserting the equality across the canonical parameter set.

Status

Resolved

[L-07] covenants/native/amm_pool_cp_v3.sil#removeLiquidity - a position whose KAS and token floors both round to zero cannot be withdrawn

Description

The `removeLiquidity` entrypoint pins both sides to their exact proportional floors at :392-395, and an earlier guard requires at least one side to be positive with `require(dKas > 0 || dToken > 0)` at :376.

When `kasReserve * dShares` is below `totalShares` and `tokenReserve * dShares` is also below `totalShares`, both floors are zero and line 376 rejects the withdrawal for every `dShares` the holder owns. The comment at :388 claims "ANY dShares now withdraws - there is no integer step that can strand a voluntary LP", which holds for each side separately but not for their conjunction.

The `addLiquidity` entrypoint has the mirror case. The `dShares` value is pinned to the floor of the proportional KAS contribution by :312-313 and must be positive by :298, so a deposit below one share-unit of KAS cannot be made at all.

Vulnerability Details

```
// covenants/native/amm_pool_cp_v3.sil:376 - at least one side must be positive
require(dKas > 0 || dToken > 0);

// covenants/native/amm_pool_cp_v3.sil:392-395 - but both are pinned to their
floors
require(dKas * totalShares <= kasReserve * dShares);
require((dKas + 1) * totalShares > kasReserve * dShares);
require(dToken * totalShares <= tokenReserve * dShares);
require((dToken + 1) * totalShares > tokenReserve * dShares);
```

Impact

An LP whose position is small enough that both floors round to zero cannot redeem it. The shares stay outstanding and inflate `totalShares` against the pool's fee-retention weighting permanently. Reaching the state needs an already-tiny position against a large `totalShares`.

Recommendation

Either allow a zero-value redemption that burns the shares back into inventory, or document a minimum viable position and have `addLiquidity` reject deposits that would create one below it.

Status

Resolved

[L-08] `covenants/native/buy_order.sil#settle` - `BuyOrder` carries no deadline and `settle` imposes no time constraint

Description

The `BuyOrder` state is four fields at `buy_order.sil:34-37`, and none of them is a deadline. The `settle` entrypoint at `:47-58` checks the slot, the active input index, the curve covid, a non-zero buyer, the payout amount and the output Kron. It imposes no time and no DAA-score bound.

The `batchBuy` entrypoint is permissionless and `settle` needs no buyer signature, so any observer may settle a funded order at any point after placement, at whatever price the curve has reached by then. The two-sided band at `curve_cp.sil:569-570` pins the batch to the constant-product price at settlement, not at placement, so `minTokenOut` is the buyer's only protection, and the curve floors it at 1 with `require(order.minTokenOut >= 1 && order.minTokenOut <= MAX_TOKEN)` at `:527`.

Vulnerability Details

```
// covenants/native/buy_order.sil:34-37 - four fields, no deadline
byte[32] curveCovid = initCurveCovid;
byte[32] buyerIdentifier = initBuyerIdentifier;
int kasIn = initKasIn;
int minTokenOut = initMinTokenOut;
```

Impact

A buyer's escrow can sit unexecuted while the curve runs up, then settle at a materially worse price than intended, bounded only by a slippage floor chosen at placement. This is the standard reason AMM routers take a deadline parameter.

Recommendation

Add a deadline field to `BuyOrder` state and enforce it in `settle` against the transaction time or a DAA-score equivalent, so an unsettled order becomes refund-only once its window closes.

Status

Acknowledged

Note

batchBuy is not enabled on the current schema. This item is deferred with the feature and carries no live exposure.

[L-09] covenants/native/kcc20.sil#transfer - the covenant-id branch has no non-zero guard on the owner, so a zero-owned balance is spendable by anyone

Description

The covenant-id branch compares the owner identifier against `OpInputCovenantId(witnesses[i])` with no guard rejecting the all-zero value. That opcode returns the zero hash for any input that is not a covenant, so a state carrying a 32-zero `ownerIdentifier` under `identifierType 0x02` is satisfied by any ordinary input. The `ZERO_COVID` constant is not declared in `kcc20.sil` at all. It exists at `amm_pool_cp_v3.sil:86` and `curve_cp.sil:86`.

The pool documents the behaviour and designed around it rather than fixing it. The `bindLp` header at `amm_pool_cp_v3.sil:455-457` reads: "a plain input satisfies `OpInputCovenantId==ZERO`, so a ZERO-owned balance is spendable - the burn was not a burn. Removing the object closes that hole at the source." The locked LP floor is therefore never minted as a token. The same behaviour is restated at `:292` and `:369`, and `covenants/native/tools/verify-zero-covid.mjs` exists to probe it.

The residual issue is a burn-address ambiguity. A zero owner under `identifierType 0x03` is a genuine burn, because it resolves to P2PK against the all-zero pubkey. The same owner under `0x02` is spendable by anyone who attaches an ordinary input. One type byte separates the two and the covenant does not distinguish them.

Vulnerability Details

```
// covenants/native/kcc20.sil:54-55 - no `o != ZERO_COVID` guard
} else if (t == IDENTIFIER_COVENANT_ID) {
    require(OpInputCovenantId(witnesses[i]) == o);
```

Impact

A balance intended as a burn but written with type `0x02` is claimable by any party who attaches an ordinary input. Nothing live carries such a state, and creating one needs a buggy or hostile client rather than an action against another holder's balance, but the covenant offers no defence if one is created.

Recommendation

Declare `ZERO_COVID` in `kcc20.sil` and add `require(o != ZERO_COVID);` to the covenant-id branch. Builder and indexer rejects for a zero `ownerIdentifier` under `0x02` cut first-party exposure without changing covenant bytes, but do not close it, because `transfer` is permissionless and a balance authored with any other tool is unaffected.

Status

Resolved

Note

This finding was reported by the Kron team on 23 August 2026

[L-10] covenants/native/kcc20.sil#transfer - the output loop never validates `identifierType`, so an out-of-range type creates an unspendable balance

Description

The output loop at :77-95 checks only the amount, with a lower bound and a minter exemption at :91 and an upper bound at :92. The `validateOutputState` call at :94 binds the output to the supplied struct and imposes no range check of its own. No `require` anywhere in the file constrains `identifierType` to 0x00 through 0x03.

The minter rule is not part of this. It is a separate loop at :101-103 that runs only when the active branch is non-minter. The `isMinter` reference on :91 is an exemption to the amount rule, not a validation of the flag.

The input dispatch falls through for anything outside the range, with `} else {` and `require(false);` at :62-63. Since `transfer` is the only entrypoint, a balance created with an out-of-range type can never be spent by any branch.

The pool cannot produce such a state, because it pins the type on every output it creates, at `amm_pool_cp_v3.sil:144`, :213, :270, :354 and :415. Direct wallet-to-wallet transfers are the exposed path.

Vulnerability Details

```
// covenants/native/kcc20.sil:91-94 - amount is bounded, identifierType is not
require(newStates[i].amount >= 1 || (newStates[i].isMinter &&
newStates[i].amount == 0));
require(newStates[i].amount <= MAX_TOKEN);
outSum = outSum + newStates[i].amount;
validateOutputState(OpCovOutputIdx(covid, i), newStates[i]);
```

Impact

A builder defect or an off-by-one in the state encoder creates a permanently unspendable balance, with no error at creation time. The failure is self-inflicted by the spender of the inputs rather than reachable against another holder.

Recommendation

Add a range check to the output loop so an invalid type fails at creation rather than at first spend, requiring `newStates[i].identifierType` to equal one of `IDENTIFIER_PUBKEY`, `IDENTIFIER_SCRIPT_HASH`, `IDENTIFIER_COVENANT_ID` or `IDENTIFIER_ADDRESS`. Mirror it as a reject for `identifierType` above `0x03` in the builder and the indexer.

Status

Resolved

Note

This finding was reported by the Kron team on 23 August 2026

[L-11] covenants/native/kcc20.sil#transfer - the witness index is unbounded and not required to be distinct

Description

Three of the four ownership branches dereference `witnesses[i]`, a spender-supplied index, at `:53`, `:55` and `:61`. The value is never bounded against `tx.inputs.length`, and the loop never requires two entries to differ.

Every other covenant in the bundle bounds its witness index before use. The pool does it three times, at `amm_pool_cp_v3.sil:163`, `:343` and `:427`, each in the form `require(<idx> < tx.inputs.length)`. The curve does it at `curve_cp.sil:184` and the escrow at `buy_order.sil:82`, both in the form `require(<idx> >= 0 && <idx> < tx.inputs.length)`. The `kcc20.sil` module is the only one that omits the check, and it is the module where the index is fully caller-chosen rather than a protocol-fixed constant.

The missing distinctness requirement is the more meaningful half. Because one index may be reused for every iteration, a single qualifying input authorizes every balance in the group that names the same owner, which is the mechanism behind the presence-ownership exposure reported separately.

Vulnerability Details

```
// covenants/native/amm_pool_cp_v3.sil:163 - the pool bounds its index
require(reserveInIdx < tx.inputs.length);

// covenants/native/kcc20.sil:55 - kcc20 does not
require(OpInputCovenantId(witnesses[i]) == o);
```

Impact

An out-of-range index depends on VM bounds behaviour at runtime, which was not exercised against a live node in this review. The repeated-index case is reachable today and is what allows one co-present input to authorize an entire group.

Recommendation

Add `require(witnesses[i] >= 0 && witnesses[i] < tx.inputs.length);` at the top of the dispatch loop, matching the form the other three covenants use, and require the indices to be strictly increasing so no single input can authorize more than one balance.

Status

Resolved

[L-12]**[kron-covenant-audit]**

covenants/native/amm_pool_cp_v3.sil#addLiquidity - the recipient of released value is not constrained

Description

A KCC-20 balance carries four state fields: `ownerIdentifier`, `identifierType`, `amount` and `isMinter`. Where the pool creates a balance for a user, three are checked and the owner is not. This applies at :212-214, :269-271, :353-355 and :414-416. In the whole covenant `ownerIdentifier` is checked once, at :143, where it pins a balance to the pool itself. For the KAS leg there is no output constraint at all. In `swapTokenForKas` and `removeLiquidity` the pool UTX0 value drops and that KAS is paid out somewhere. The only four `scriptPubKey` constraints in the covenant are the fee outputs at :227, :229, :280 and :282.

Sequence:

1. A deposit is requested through the front end.
2. The front end assembles the transaction and sets `lpSharesOut.ownerIdentifier` to an address of its own choosing.
3. The wallet displays the KAS movements. The token movements are held in the covenant redeem script and are not decoded.
4. The user signs.
5. The covenant checks amount, identifier type and minter flag. All three pass.
6. The LP shares are created at the chosen address.

The deposit case carries the largest loss. The user's KAS and tokens are already committed to the pool reserve, and the LP shares are the only object that can withdraw them. The holder input in `removeLiquidity` is checked at :434-436 for type, amount and minter flag, and not for owner, so the shares are redeemable by whoever holds them. A user signature is required on every path, so the pool cannot be drained by a party outside the transaction.

Vulnerability Details

```
// covenants/native/amm_pool_cp_v3.sil:353-355 - amount, type and minter flag,  
no owner  
require(lpSharesOut.amount == dShares);  
require(lpSharesOut.identifierType == IDENTIFIER_ADDRESS);  
require(!lpSharesOut.isMinter);  
  
// covenants/native/amm_pool_cp_v3.sil:143 - the only ownerIdentifier check in  
the covenant  
require(token.ownerIdentifier == self);
```

Impact

A trader's bought tokens, a seller's proceeds, or a liquidity provider's shares and withdrawal are directed to an address chosen by the transaction builder. In the deposit case the loss is the whole deposit rather than the amount in flight. The pool's own accounting remains correct, so no on-chain signal is produced.

Recommendation

Take the expected recipient as an entrypoint argument and pin it on every presence-owned output, adding `require(lpSharesOut.ownerIdentifier == <recipient>);` and the equivalent for `traderTokenOut`, `traderChangeOut` and `lpTokenOut`. Pin the released KAS the same way the fee legs are pinned, with a `ScriptPubKeyP2PK` requirement on the output receiving `kasOut` in `swapTokenForKas` and `dKas` in `removeLiquidity`.

Status

Resolved

[L-13] [kron-covenant-audit] covenants/native/kcc20.sil#transfer - input amounts are bounded from below only

Description

The input loop bounds each amount only from below, at :71. The output loop bounds each amount from both sides, at :91 and :92, with a maximum of MAX_TOKEN. Conservation is enforced at :100.

The amount field is a signed 64-bit integer with a maximum of approximately 9.223e18. The maxIns value is 4 by default and may be set as high as 8 by boundedIo at backend/nativeTemplates.mjs:106-110. A set of large positive inputs can therefore carry inSum past the integer maximum and wrap it to a small value chosen to match the declared outSum. The comment at :65-70 states that checking inputs directly makes conservation locally provable without relying on the lineage invariant. Both bounds are required for that property to hold.

A balance above MAX_TOKEN can originate only at genesis. Every balance created by transfer is already capped by the output loop, and a genesis output is created in a transaction where no input carries the covenant id, so kcc20 does not execute and the amount is not checked. For a KRON-native token the amounts of both genesis outputs are pinned by curve_cp.init. For an externally imported token, supported per web/src/externalPool.ts and covenants/native/tools/verify-standalone-pool.mjs, no equivalent check exists.

Sequence:

1. A token is deployed outside KRON carrying a genesis balance above MAX_TOKEN.
2. The token is listed in a standalone pool.
3. Liquidity is added by other parties.
4. A transfer is submitted combining oversized inputs so that inSum wraps to the declared outSum.
5. Balances below MAX_TOKEN are produced that no supply backs.
6. Those balances are folded into the pool through swapTokenForKas, bounded per trade by :248, :253 and the constant-product invariant.
7. The sequence is repeatable.

Where the token carries a live mint branch the sequence is unnecessary, as supply may be minted directly. The condition therefore applies to a token presenting as having renounced mint authority, where the indexer reports a fixed supply.

Vulnerability Details

```
// covenants/native/kcc20.sil:71-72 - lower bound only
require(a >= 0);
inSum = inSum + a;

// covenants/native/kcc20.sil:92 - the outputs do get an upper bound
require(newStates[i].amount <= MAX_TOKEN);
```

Impact

The KAS reserve of a pool holding an externally imported token is drained against balances that no supply backs. Liquidity providers and KAS holders in that pool bear the loss. No exposure exists on a KRON-native token. The condition is not visible in the indexer, which reports the token as having a fixed supply.

Recommendation

Add `require(a <= MAX_TOKEN);` beside the existing non-negativity guard, so the input sum is bounded by `maxIns * MAX_TOKEN` and conservation becomes locally provable as the comment intends. Independently, reject an imported token at registration if any genesis output carries an amount above `MAX_TOKEN`.

Status

Resolved

QA

[Q-01] [covenants/native/curve_cp.sil#soleCurveContinuation](#) -

`soleCurveContinuation()` [constrains only the output side of the C group](#)

Description

Every trading entryptpoint starts by calling `soleCurveContinuation()`, which proves the transaction produces exactly one curve continuation output at output 0, preventing the curve from forking into multiple UTXOs.

However, it never checks the input side (`OpCovInputCount(self) == 1` is missing).

With that said, the deploy infrastructure creates exactly one curve output ([web/src/native/curveCpTx.ts:167-182](#)), and `init`'s `soleCurveContinuation()` + full-output scan prevents duplicate covenant IDs within the `init` tx. A second C-bearer is not reachable in the current codebase.

Recommendation

Add `require(OpCovInputCount(self) == 1);` to `soleCurveContinuation()`.

Status

Resolved

[Q-02] `web/src/native/curveCpTx.ts#batchBuy` - Two-sided price band + pro-rata allocation are covenant-only, not builder-validated

Description

The `batchBuy` covenant carries a two-sided CP price band (`kNew >= kFloor` and `kNew <= kFloor + q`, lines 566-570) and a pro-rata buyer allocation (lines 597-609).

The first-party builder, however:

1. Uses caller-supplied `order.tokenOut` and validates only the floor half of the band (`web/src/native/curveCpTx.ts:378`); the ceiling is not checked.
2. Does not compute pro-rata allocation; it trusts the caller's `tokenOut` per order.
3. Is not routed to at all; the comment at `web/src/native/curveCpTx.ts:359-365` states `ADAPTIVE_BATCH_SCHEMA_CURRENT` is withheld from `batchCapable` in `web/src/native/tradeCpFlow.ts`.

The covenant enforces both the ceiling and pro-rata on-chain, so any mismatch results in a VM-level revert. The consequence is purely a build-time UX issue: the two-sided band is currently dead code on the first-party path, and a builder that constructs a batch violating the ceiling or pro-rata will produce transactions that fail at the VM rather than being rejected at build time.

Recommendation

If `batchBuy` is ever re-enabled, the builder should compute the authoritative allocation (`ceil(kFloor/q)`) and validate the ceiling, as the reverted code already does.

Note

The Kron team clarified that the builder's caller-supplied `tokenOut` is a deliberate, in-code-documented design. The covenant enforces the two-sided aggregate band and the pro-rata split on-chain, so any settler (first-party or external) is bound. The first-party client cannot route to `batchBuy` on the current schema (`ADAPTIVE_BATCH_SCHEMA_CURRENT` is withheld from `batchCapable`). The authoritative `ceil(kFloor/q)` allocation will return together with the allowlist entry once a batch has settled on a real chain; the two changes are coupled by the in-source notes at both sites.

Status

Acknowledged

Note

batchBuy is not enabled on the current schema. This item is deferred with the feature and carries no live exposure.

[Q-03] covenants/native/buy_order.sil#settle - settle's slot is redundant with `this.activeInputIndex`

Description

`slot` is passed as a witness argument but is redundant with `this.activeInputIndex` (the order's own input index).

Recommendation

Derive `slot` from `this.activeInputIndex` and remove the witness parameter.

Status

Resolved

[Q-04] covenants/native/curve_cp.sil#batchBuy - Permanent dust lock-up**Description**

`count >= 2` (line 505) makes a lone order refund-only, which is by design: relaxing to `count >= 1` would put a single-order batch at 2 covid-A outputs, colliding with `buy`'s cardinality and breaking the settle-window disjointness that `settle` relies on.

Recommendation

Document as a known design choice. The `count >= 2` constraint is load-bearing for the settle window's safety argument.

Note

The Kron team confirmed this is deliberate. Each buyer output's dust is pre-funded by its own order (the `ORDER_TOKEN_DUST` term of the input-value equation), and the inventory continuation is funded by the carrier input, not by buyer escrow.

Status

Acknowledged

Note

`batchBuy` is not enabled on the current schema. This item is deferred with the feature and carries no live exposure.

[Q-05] `covenants/native/curve_cp.sil#buy` - `buy/sell/graduate` don't check `!invIn.isMinter` on the inventory input

Description

`buy/sell/graduate` check `invIn.ownerIdentifier`, `invIn.identifierType`, and `invIn.amount` on the inventory input, but not `!invIn.isMinter`. Only `batchBuy` checks all four (line 514).

Recommendation

Add `require(!invIn.isMinter)` to all three for consistency.

Status

Resolved

[Q-06] covenants/native/buy_order.sil#settle - settle relies on off-chain C-singularity + output-count fingerprint, not local proofs

Description

settle delegates most of its security to the curve's batchBuy. It only verifies the buyer's own payout (recipient, minimum amount, Kron shape).

Two things it cannot prove locally:

1. That the curve script ran on input 0: `require(0pInputCovenantId(0) == curveCovId)` (line 50) only proves input 0 carries covenant ID C, not that a covenant script executed on it.

This is closed by C-singularity: exactly one UTXO on-chain carries C (the real curve UTXO). A second C-bearer would require a bug in the funding/deploy transaction or a Kaspas consensus change; neither is present in the current codebase.

2. Which token the payout is in. `tokenCovId` is read from input 1 unauthenticated (line 55), and the payout is validated only against the identity-free KCC-20 Kron (line 57).

This is closed by the output-count fingerprint: settle checks `0pCovOutputCount(tokenCovId) ∈ [3,5]` (line 56), and only batchBuy (with count `∈ [2,4] → count+1 = 3-5` token outputs) lands in that range. Every archived schema caps sell at 1-2 and buy/graduate at 1-2, so no other entrypoint enters the window today.

Both properties are emergent agreements between two files (`buy_order.sil` and `curve_cp.sil`), not local proofs that settle can verify. This is documented explicitly in the covenant header (lines 6-21), and the on-chain fix was deferred because it changes the state layout.

Recommendation

If a future schema change widens any entrypoint's token-output count into the `[3,5]` range, re-derive the fingerprint argument against every archived schema before

shipping. The complete fix (binding `tokenCovid` in the `BuyOrder` constructor state) remains a deliberate, separately scoped change.

Note

The Kron team acknowledged this as a deliberate, documented design and has converted the two-file agreement from prose into Cl. A new gate (`verify:batch-covid-cardinality`) re-derives all four curve entrypoint cardinalities and the settle window from the sources, requires the window to intersect only `batchBuy` and to `equal` its interval exactly, and carries in-gate mutation controls. Widening either side now fails the build until the argument is re-derived. Re-deriving the argument against the archived schemas on any future widening remains a stated manual step (every archived non-batch entrypoint caps at 2 or fewer covid-A outputs today). The complete on-chain fix (carrying `tokenCovid` in order state) remains deferred as the header records: a state-layout change, separately scoped.

Status

Acknowledged

Note

`batchBuy` is not enabled on the current schema. This item is deferred with the feature and carries no live exposure.

[Q-07] `covenants/native/curve_cp.sil#buy` - `buy/sell` rely on the `nu <= MAX_UNITS` check to prevent `newKas` overflow as a side effect

Description

In `buy`, the new KAS reserve is computed as `int newKas = realKas + kasIn` (line 270), and only afterward is it bounded with `require(newKas <= MAX_KAS)` (line 275). If `kasIn` is near `int64` max, the addition overflows before the bound is checked.

However, the overflow is not exploitable because a separate computation at line 276-277 catches it as a side effect: `nu = ru + ki` (where `ru = realKas/SCALE` and `ki = kasIn/SCALE`) is checked against `MAX_UNITS = 9e8`, which implicitly bounds `kasIn` to `~9e14`, far below the `int64` overflow threshold. Since `nu` itself doesn't overflow, the compiler cannot fold that check.

Recommendation

Bound `kasIn` before the addition, matching `batchBuy`'s pattern:

```
require(kasIn <= MAX_KAS - realKas); // buy, before line 270
require(kasOut <= realKas);          // sell, before line 355
```

This makes the overflow unreachable by construction, rather than relying on a downstream check's side effect.

Status

Resolved

[Q-08] `covenants/native/curve_cp.sil#batchBuy` - the allocation assertion is satisfied by construction

Description

The last slot is assigned the remainder at :602, before `allocated` is updated on the next line. The assertion `require(allocated == totalTokenOut)` at :610 therefore holds unconditionally for any `count` of 2 or more. It reads as a conservation proof and proves nothing.

The real guarantees are the per-order floor `require(share >= order.minTokenOut)` at :604 and the two-sided price band at :569-570.

Vulnerability Details

```
// covenants/native/curve_cp.sil:601-603 - the last slot is defined as the remainder
int share = totalTokenOut * (order.kasIn / SCALE) / totalUnits;
if (i == count - 1) { share = totalTokenOut - allocated; }
allocated = allocated + share;

// covenants/native/curve_cp.sil:610 - so this cannot fail
require(allocated == totalTokenOut);
```

Impact

None. The assertion is dead rather than wrong, but a reader may take it for an enforced conservation check.

Recommendation

Assert the pre-remainder sum instead, or annotate the line as a documentation aid so it is not mistaken for an active constraint.

Status

Resolved

[Q-09] `covenants/native/curve_cp.sil#batchBuy` - the smallest permitted order escrows 1.8 KAS to buy 0.5 KAS of token

Description

Each of the three fee legs is floored to FEE_OUT_MIN at :531-533, then the escrow is pinned to their sum at :534.

The constants are MIN_ORDER_KAS at 0.5 KAS (:116), ORDER_TOKEN_DUST at 0.5 KAS (:117), FEE_OUT_MIN at 0.2 KAS (:118) and ORDER_TX_FEE at 0.2 KAS (:136). At the minimum order the escrow is therefore 0.5 plus 0.5 plus three legs of 0.2 plus 0.2, which is 1.8 KAS to buy 0.5 KAS of token.

The token dust returns to the buyer as the carrier value of their token UTXO, so it is not all lost, but the fee floors alone are 0.8 KAS against a 0.5 KAS order.

Vulnerability Details

```
// covenants/native/curve_cp.sil:531-534
if (creatorFee < FEE_OUT_MIN) { creatorFee = FEE_OUT_MIN; }
if (platformFee < FEE_OUT_MIN) { platformFee = FEE_OUT_MIN; }
if (devFundFee < FEE_OUT_MIN) { devFundFee = FEE_OUT_MIN; }
require(tx.inputs[2 + i].value == order.kasIn + ORDER_TOKEN_DUST + creatorFee +
platformFee + devFundFee + ORDER_TX_FEE);
```

Impact

None on protocol safety. Small orders are economically unattractive in a way that is not surfaced to users before they commit funds.

Recommendation

Put the effective minimum order cost in user-facing copy, and consider retuning FEE_OUT_MIN against MIN_ORDER_KAS so the floor legs do not dominate a minimum-size order.

Status

Resolved

[Q-10] covenants/native/amm_pool_cp_v3.sil#solePoolContinuation - the continuation helper does not reject a zero self

Description

The helper returns the pool's own covenant id, which requirePoolOwnedNonMinter at :142-146 then uses as the ownerIdIdentifier of every pool-owned balance the entrypoint validates. It never compares self against ZERO_COVID, declared at :86. The curve's soleCurveContinuation() at curve_cp.sil:167-172 is byte-identical and has the same gap. Reading KIP-20 section 5.3, a covenant output set for a given id counts only outputs whose authorizing input already carries that id, and a UTXO carrying the zero id cannot be created, so the count of the zero id is always 0 and require(OpCovOutputCount(self) == 1) rejects a zero self before any balance is validated. That argument rests on the consensus specification rather than on anything provable from the covenant sources, and no test under covenants/native/tools exercises a zero-id active input against the helper.

The covenant elsewhere treats a zero covenant id as actively dangerous, guarding against it at :295, :372 and :472, which makes the omission here inconsistent rather than deliberate.

Vulnerability Details

```
// covenants/native/amm_pool_cp_v3.sil:135-140 - no `self != ZERO_COVID` guard
function solePoolContinuation(): byte[32] {
  byte[32] self = OpInputCovenantId(this.activeInputIndex);
  require(OpCovOutputCount(self) == 1);
  require(OpCovOutputIdx(self, 0) == SELF_OUT);
  return self;
}
```

Impact

None on the current schema. The guard is absent rather than exploitable.

Recommendation

Add `require(self != ZERO_COVID);` to the helper as a local assertion, so the property does not depend on a rule stated only in an external specification. Add a regression case exercising a zero-id active input.

Status

Resolved



Centralisation

The audited covenants have no privileged roles.

All entrypoints are permissionless; the protocol has no owner-gated functions.



Conclusion

After Hashlock's analysis, the Kron project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities. Hashlock also recommends a re-review once silverc reaches 1.0 and supporting tooling becomes available.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the covenant under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the covenant details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these covenants in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the covenant source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the covenant. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this covenant.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Covenants are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the covenant can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited covenants.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.